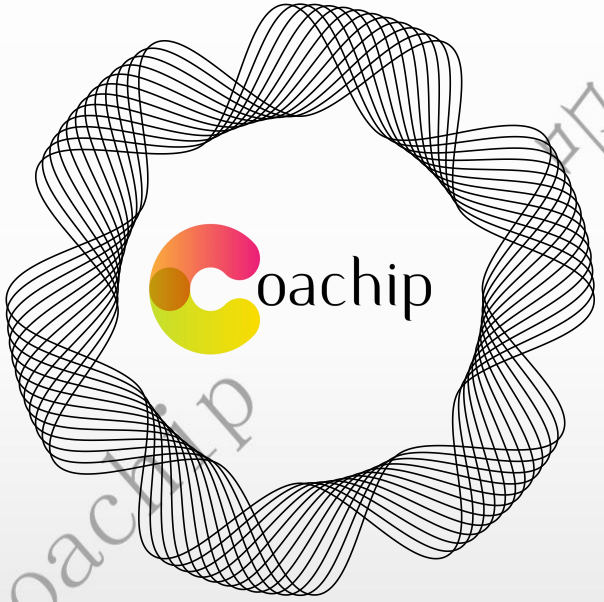


SV过程语句和子程序

叩特电子Coachip

课程内容及目标

课程内容：

- 硬件的过程语句
- 软件的过程语句

目标：

- 了解过程语句及使用



叩特电子Coachip

如何区分always和initial

需要清楚哪些语句应该被放置于硬件世界，哪些程序应该被放置于软件世界。

module/endmodule，interface/endinterface可以被视为硬件世界

program/endprogram，class/endclass可以被视为软件世界。

叩特电子Coachip

如何区分always和initial（续）

- always是为了描述硬件的行为，在使用时需要注意：哪种使用方式是时序电路描述，哪种使用方式是组合电路描述；
- always中的@(event..)敏感列表是为了模拟硬件信号的触发行为，需要正确对标硬件行为和always过程块描述。需要理解硬件行为的核心要素有哪些；
- always过程块是用来描述硬件时序电路和组合电路的正确打开方式，因此只可以在module或者interface中使用。

即特电子Coachip

如何区分always和initial（续）

- initial从名字也可以看得出来，与always在执行路径上有明显区别，即initial非常符合软件的执行方式，即只执行一次。
- initial和always一样，无法被延迟执行，即在仿真一开始它们都会同时执行，不同initial和always之间在执行顺序上是没有顺序可言的，因此不应该将它们在代码中的前后顺序与它们的执行顺序画上等号。
- initial从其执行路径的属性来看，它不应该存在于硬件设计代码中，它本身不可综合，对于描述电路没有任何帮助。
- initial就是为了测试而生的，由于测试需要按照时间顺序的习惯即软件方式来完成，所以initial便可以实现这一要求。

即特电子Coachip

SV的过程语句和子程序 – 硬件的过程语句

- 在Verilog时代，所有的测试语句都可以被放置在initial中，为了便于统一管理测试顺序，建议将有关测试语句都放置在同一个initial过程块中。
- initial过程块可以在module、interface和program中使用。
- 对于过程块的书写方式，请记住用begin..end将其作用域‘包’住，这一建议同样适用于稍后提到的控制语句、循环语句等等，可以将其对应于C语言中的花括号{}，方便记忆。

即特电子Coachip

软件的过程语句—函数function

SV函数定义同C语言非常类似

- 可以在参数列表中指定输入参数（input）、输出参数(output)、输入输出参数（inout）或者引用参数(ref)。
- 可以返回数值或者不返回数值（void）。

```
function int double(input int a);
  return 2*a;
endfunction
initial begin
      $display("double of %0d is %0d", 10, double(10));
end
```

叩特电子Coachip

软件的过程语句—函数function（续）

SV函数也有自己的特点:

- 默认的数据类型是为logic，例如 input [7:0] addr。
- 数组可以作为形式参数传递。
- function可以返回或者不返回结果，如果返回即需要用关键词return，如果不返回则应该在声明function时采用void function()。
- 只有数据变量可以在形式参数列表中被声明为ref类型，而线网类型则不能被声明为ref 类型。
- 在使用ref 时，有时候为了保护数据对象只被读取不被写入，可以通过const的方式来限定ref 声明的参数。
- 在声明参数时，可以给入默认数值，例如 input [7:0] addr = 0，同时在调用时如果省略该参数的传递，那么默认值即会被传递给function。

软件的过程语句—任务task

任务相比于函数要更加灵活，且有以下不同点：

- task无法通过return返回结果，因此只能通过output、inout或者ref的参数来返回。
- task内可以置入耗时语句，而function则不能。常见的耗时语句包括@event、wait event、# delay等。

```
task mytask1 (output logic [31:0] x, input logic y);
   ...
endtask
```

叩特电子Coachip

function和task的使用

通过上面的学习比较，对function和task建议的使用方式是：

- 对于初学者，傻瓜式用法即全部采用task来定义方法，因为它可以内置常用的耗时语句。
- 对于有经验的使用者，请今后对这两种方法类型加以区别，在非耗时方法定义时使用function，在内置耗时语句时使用task。这么做的好处是在遇到了这两种方法定义时，就可以知道**function只能运用于纯粹的数字或者逻辑运算，而task则可能会被运用于需要耗时的信号采样或者驱动场景。**
- function和task均可调用function；建议task只调用task，这是因为如果被调用的task内置有耗时语句，则外部调用它的方法类型必须为task。

即特电子Coachip

wait和@的区别

```
program automatic test_event();
    logic clk,a,b,c;
    initial begin
        clk =1'b1;
    end

    initial begin
        wait(clk)
        $display("i have reached clk");
    end

    initial begin
        @(clk)
        $display("reached posedge of clk");
    end
endprogram
```

因为clk一直没有变，@操作符无效，但是wait只需要clk为高即可触发，仿真结果如下：

```
i have reached clk
```

叩特电子Coachip

wait和@的区别

```
program automatic test_event();
    logic clk,a,b,c;
    initial begin
        clk =1'b1;
        #3 clk = 1'b0;
    end
    initial fork
        begin
            #5 ;
            wait(clk)
            $display("i have reached clk");
        end

        begin
            @(clk)
            $display("reached negedge of clk");
        end
    join
endprogram
```

因为过了5个时间单位后，clk为低，此时wait无效，因此只会出现第二种情况，仿真结果如下：

```
reached negedge of clk
```

叩特电子Coachip

wait和@的区别

```
program automatic test_event();
    logic clk,a,b,c;
    initial begin
        clk =1'b0;
        #3 clk = 1'b1;
    end
    initial fork
        begin
            #5 ;
            wait(clk)
            $display("i have reached clk");
        end

        begin
            @(clk)
            $display("reached posedge of clk");
        end
    join
endprogram
```

此时两种状态都有效，但是3个时间单位时上升沿触发，因此第二种情况先出现，第一种情况晚出现，仿真结果如下：

```
reached posedge of clk
i have reached clk
```

叩特电子Coachip

wait和@的区别

```
program automatic test_event();
    logic clk,a,b,c;
    initial begin
        clk =1'b0;
        #3 clk = 1'b1;
    end
    initial fork
        begin
            #5 ;
            wait(clk.triggered())
            $display("i have reached clk");
        end

        begin
            @(clk)
            $display("reached posedge of clk");
        end
    join
endprogram
```

会报错，因为clk不是 event类型的变量，不能用triggered函数

叩特电子Coachip

SV的过程语句和子程序 – 本章回顾

- 硬件的过程语句 always
- 软件的过程语句 initial / function / task
- wait和@的区别

叩特电子Coachip

Q&A

昂特电子Coachip